

Propositional Logic

Propositional Logic - Syntax, Semantics and Inferences

Luciano Serafini – Maurizio Lenzerini

FBK, Trento, Italy – Sapienza Università di Roma

Revised and adapted by F. Patrizi

Formal logical system

A **formal logical system** is constituted by several components, including:

- The **language**, with its
 - ▶ syntax (how to form sentences)
 - ▶ semantics (how to assign meaning to sentences)
 - ▶ pragmatics (how to use the logic for a specific objective)

that enables forming the sentences that are acceptable in the formal system

- The **inferencing** apparatus, that enables studying all the relevant relationships between sentences of the language

Propositional logic - Intuition

The first formal logical system that we study is **propositional logic**.

- Propositional logic is the logic of **propositions**
- Intuitively, a proposition is anything that has a truth value (i.e., can be **true** or **false**) in a state of the world (context) of interest.
- The same proposition can be expressed in different ways.

E.g.

- ▶ “B. Obama is drinking a beer”
- ▶ “ The U.S.A. president is drinking a beer”, and
- ▶ “B. Obama si sta facendo una birra”

express the same proposition.

- Warning: obviously, not all sentences in natural language are propositions!
- The language of propositional logic allows us to use propositions and to compose complex sentences (formulas) on the basis of propositions and connectives.

Syntax of propositional logic language: the alphabet

To define the syntax of the propositional logic language, we define the alphabet (set of atomic symbols that can be used in the language), and then the set of sentences that can be formed using the alphabet.

Definition (Propositional alphabet)

Non logical symbols A countable set $\mathcal{P}$ of symbols called **propositional variables**, or **propositional atoms**

Logical symbols (connectives) $\neg, \wedge, \vee, \supset, \rightarrow$ and $\equiv$

Separator symbols "(" and ")"

The intuitive meaning of the connectives are as follows:

- $\neg$ is the negation symbol: $\neg A$ has the opposite truth value as A
- $\wedge$ is the conjunction symbol: $A \wedge B$ is true exactly when both A and B are true
- $\vee$ is the disjunction symbol: $A \vee B$ is true exactly when at least one between A and B is true
- both $\supset$ and $\rightarrow$ denote the material implication symbol: $A \rightarrow B$ means that whenever A is true, B is true as well
- $\equiv$ is the material equivalence symbol: $A \equiv B$ is true if A and B have the same truth value.

The notion of inductive definition

In what follows, we will often define sets and functions **inductively**. An inductive definition of a set S specifies which are the elements of S by using

- one or more **base steps**, that directly specify that some elements belong to S
- one or more **inductive steps**, each of the form **if $x \in S$, then $y \in S$** , where y is related somehow with x .
- a **closure** condition, asserting that nothing else is in S (i.e., anything not deriving from the base or inductive steps is not in S)

Whenever we indicate that a certain definition is inductive, its closure condition is implicit, and therefore we can avoid specifying such condition explicitly.

Example of inductive definition

Example

The set Nat of natural numbers is defined **inductively** as follows:

- ① $0 \in Nat$
- ② if $x \in Nat$ then $s(x) \in Nat$

Note that with the inductive definition of Nat , we can also

- define inductively any function F of the form $F : Nat \longrightarrow C$, using the following scheme:
 - ① **Base step** We directly provide the definition of $F(0)$
 - ② **Inductive step** Starting from $F(x)$, we provide the definition of $F(s(x))$
- prove that a certain property T holds for **every** natural numbers, using the following scheme:
 - ① **Base step** We directly prove that $T(0)$ holds
 - ② **Inductive step** Based on the assumption that $T(x)$ holds, we provide the proof that $T(s(x))$ holds as well

Syntax of propositional logic language: the formulas

Definition (Well formed formulas (or simply formulas))

A propositional logic formula on the alphabet $\mathcal{P}$ is a finite expression built according to the following **inductive** definition:

- every $P \in \mathcal{P}$ is an **atomic formula**, and every atomic formula is a **formula**
- if A and B are formulas, then the following are **formulas**:
 - ▶ (A)
 - ▶ $\neg A$
 - ▶ $A \wedge B$
 - ▶ $A \vee B$
 - ▶ $A \supset B$ (also written as $A \rightarrow B$)
 - ▶ $A \equiv B$

Example (formulas and non formulas)

Formulas	Non formulas
$P \rightarrow Q$	PQ
$P \rightarrow (Q \rightarrow R)$	$(P \rightarrow \wedge((Q \rightarrow R)$
$P \wedge Q \rightarrow R$	$P \wedge Q \rightarrow \neg R \neg$

Reading formulas

Problem

How do we read the formula $P \wedge Q \rightarrow R$?

The formula $P \wedge Q \rightarrow R$ can be read in two ways:

- ❶ $(P \wedge Q) \rightarrow R$
- ❷ $P \wedge (Q \rightarrow R)$

Symbol priority

$\neg$ has higher priority, then $\wedge$, $\vee$, $\rightarrow$ and $\equiv$. Parenthesis can be used around formulas to stress or change the priority.

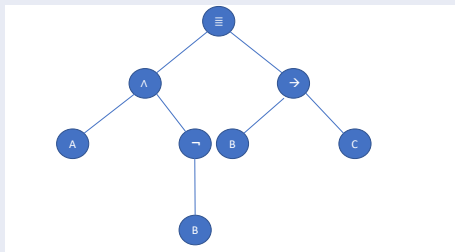
Symbol	Priority
$\neg$	1
$\wedge$	2
$\vee$	3
$\rightarrow$	4
$\equiv$	5

Formulas as trees

Tree form of a formula

A formula can be seen as a tree. Leaf nodes are associated to propositional variables, while intermediate (non-leaf) nodes are associated to connectives.

For instance the formula $(A \wedge \neg B) \equiv (B \rightarrow C)$ can be represented as the tree



Subformulas

Definition ((Proper) Subformula)

- A is a **subformula** of itself
- A, B are **subformulas** of $A \wedge B, A \vee B, A \supset B, A \rightarrow B, A \equiv B$
- A is a subformula of $\neg A$, and A is a subformula of (A)
- if A is a subformula of B and B is a subformula of C , then A is a subformula of C .
- A is a **proper subformula** of B if A is a subformula of B and A is different from B .

Remark

The subformulas of a formula represented as a tree correspond to all the different subtrees of the tree associated to the formula, one for each node.

Subformulas

Example

The subformulas of $(p \rightarrow (q \vee r)) \rightarrow (p \wedge \neg p)$ are

$$\begin{aligned} & (p \rightarrow (q \vee r)) \rightarrow (p \wedge \neg p) \\ & (p \rightarrow (q \vee r)), \quad (p \wedge \neg p), \quad p \rightarrow (q \vee r), \quad (q \vee r) \\ & p \wedge \neg p, \quad q \vee r \\ & p, \quad \neg p, \quad q, \quad r \end{aligned}$$

Proposition

Every formula has a finite number of subformulas

Note that the above is a **meta-sentence**, i.e., a sentence telling something about the formal logical system. In general we should prove that a proposition (a meta-sentence) is true. When we do so, we use logic for proving properties of logic!

Propositional logic language: semantics

How can we specify the meaning of the sentences of propositional logic?

- The simplicity of the propositional language leads naturally to deciding that the meaning of any sentence coincides with its truth value, i.e., is either true or false.
- But, for example, a sentence like $P_1 \wedge P_2$ is either true or false **depending** of the truth value of P_1 and P_2 . So, the meaning of a sentence depends on the truth value of the propositional variables in it!
- In turn, the truth value of a propositional variable depends on the specific intended meaning of the variable in the considered context. For example, if “piove” is a propositional variable, its truth value depends on the “world” (context, i.e., time and space) that we are considering.
- In order to capture the notion of world (or, context), propositional logic introduces the notion of **interpretation**.

Propositional logic language: semantics

The notion of interpretation is central to the semantics of propositional logic.

Definition (Propositional interpretation (or simply interpretation))

Given a propositional alphabet with propositional variables $\mathcal{P}$, a **propositional interpretation** is a function $\mathcal{I} : \mathcal{P} \rightarrow \{\text{True}, \text{False}\}$

Remark

A propositional interpretation can be actually based on the subset T of $\mathcal{P}$ containing exactly the propositional variables that are true in the context corresponding to the interpretation. From this point of view, the interpretation $\mathcal{I}$ can be thought of as the **characteristic function** of T , i.e., the function satisfying the condition $\mathcal{I}(P) = \text{True}$ if and only if $P \in T$.

Interpretation in propositional logic: example

Consider the following sentence over the alphabet $\{p, q, r\}$:

$$(p \wedge (\neg q \vee r)) \vee (p \wedge r)$$

Some possible interpretations for the above sentence:

Example

	p	q	r	Set theoretic representation
$\mathcal{I}_1$	True	True	True	$\{p, q, r\}$
$\mathcal{I}_2$	True	False	True	$\{p, r\}$
$\mathcal{I}_3$	True	False	False	$\{p\}$
$\mathcal{I}_4$	False	False	False	$\{\}$

In the above picture, the table should be read as follows: $\mathcal{I}_1$ is the interpretation function such that $\mathcal{I}_1(p) = \text{True}$, $\mathcal{I}_1(q) = \text{True}$, $\mathcal{I}_1(r) = \text{True}$, and so on.

Evaluation of a propositional formula in an interpretation

We now illustrate the rules for deciding whether the formula A is true in the interpretation $\mathcal{I}$ (or, equivalently, whether $\mathcal{I}$ satisfies A , written as $\mathcal{I} \models A$).

Definition ($\mathcal{I} \models A$)

$\mathcal{I} \models A$ is inductively defined as follows:

- If $P \in \mathcal{P}$, $\mathcal{I} \models P$ if $\mathcal{I}(P) = \text{True}$
- $\mathcal{I} \models (A)$ if $\mathcal{I} \models A$
- $\mathcal{I} \models \neg A$ if it is not the case that $\mathcal{I} \models A$ (also written $\mathcal{I} \not\models A$)
- $\mathcal{I} \models A \wedge B$ if both $\mathcal{I} \models A$ and $\mathcal{I} \models B$
- $\mathcal{I} \models A \vee B$ if $\mathcal{I} \models A$ or $\mathcal{I} \models B$
- $\mathcal{I} \models A \rightarrow B$ if it is not the case that $\mathcal{I} \models A$ and $\mathcal{I} \not\models B$
- $\mathcal{I} \models A \equiv B$ if both $\mathcal{I} \models A \rightarrow B$ and $\mathcal{I} \models B \rightarrow A$

Evaluation of a propositional formula: example

Example

Let $\mathcal{I} = \{P\}$, and let us check if $\mathcal{I} \models (P \wedge Q) \vee (R \rightarrow S)$.

We replace each occurrence of each primitive proposition of the formula with the truth value assigned by $\mathcal{I}$, and apply the definition for connectives.

$$(True \wedge False) \vee (False \rightarrow False) \quad (1)$$

$$False \vee True \quad (2)$$

$$True \quad (3)$$

Observations on connectives

- $A \rightarrow B$ has the same meaning as $\neg A \vee B$
- $\neg(A \vee B)$ has the same meaning as $\neg A \wedge \neg B$ (De Morgan law 1)
- $\neg(A \wedge B)$ has the same meaning as $\neg A \vee \neg B$ (De Morgan law 2)
- $A \rightarrow B$ has the same meaning as $\neg B \rightarrow \neg A$
- $A \rightarrow \neg B$ has the same meaning as $B \rightarrow \neg A$
- A is a sufficient condition for B can be written as $A \rightarrow B$, and can read as “if A then B ”
- A is a necessary condition for B can be written as $B \rightarrow A$
- A is a necessary and sufficient for B can be written as $A \equiv B$, and can be read as “ A if and only if B ”
- **NAND** is another connective that is defined as $A \text{ NAND } B$ is defined as $\neg(A \wedge B)$
- **NOR** is another connective that is defined as $A \text{ NOR } B$ is defined as $\neg(A \vee B)$

Exercise

Prove the following proposition (i.e., prove that the following meta-sentence is true).

Proposition

Let $\mathcal{I}$ and $\mathcal{I}'$ be two interpretations for a formula A . If for every propositional variable P appearing in A it holds that $\mathcal{I}(P) = \mathcal{I}'(P)$, then $\mathcal{I} \models A$ if and only if $\mathcal{I}' \models A$.

Algorithm for checking if $\mathcal{I} \models A$

Lazy evaluation algorithm (1/2)

$(A \text{ is } p)$	<pre>check($\mathcal{I} \models p$) if $\mathcal{I}(p) = \text{true}$ then return YES else return NO</pre>
$(A \text{ is } B \wedge C)$	<pre>check($\mathcal{I} \models B \wedge C$) if check($\mathcal{I} \models B$) then return check($\mathcal{I} \models C$) else return NO</pre>
$(A \text{ is } B \vee C)$	<pre>check($\mathcal{I} \models B \vee C$) if check($\mathcal{I} \models B$) then return YES else return check($\mathcal{I} \models C$)</pre>

Checking if $\mathcal{I} \models A$

Lazy evaluation algorithm (2/2)

$(A \text{ is } B \rightarrow C)$

```
check( $\mathcal{I} \models B \rightarrow C$ )  
  if check( $\mathcal{I} \models B$ )  
    then return check( $\mathcal{I} \models C$ )  
  else return YES
```

$(A \text{ is } B \equiv C)$

```
check( $\mathcal{I} \models B \equiv C$ )  
  if check( $\mathcal{I} \models B$ )  
    then return check( $\mathcal{I} \models C$ )  
  else return not(check( $\mathcal{I} \models C$ ))
```

Question: which is the computational complexity of the algorithm?

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"
- ② "*if Paola is happy, then she paints a picture*"
- ③ "*Paola is happy only if she paints a picture*"

The precision of formal languages avoids the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- 1 "*if Paola is happy and paints a picture then Renzo isn't happy*"

$$p \wedge q \rightarrow \neg r$$

- 2 "*if Paola is happy, then she paints a picture*"

- 3 "*Paola is happy only if she paints a picture*"

The precision of formal languages avoid the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"

$$p \wedge q \rightarrow \neg r$$

- ② "*if Paola is happy, then she paints a picture*"

$$p \rightarrow q$$

- ③ "*Paola is happy only if she paints a picture*"

The precision of formal languages avoid the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"

$$p \wedge q \rightarrow \neg r$$

- ② "*if Paola is happy, then she paints a picture*"

$$p \rightarrow q$$

- ③ "*Paola is happy only if she paints a picture*"

$$\neg(p \wedge \neg q) \text{ which is equivalent to } p \rightarrow q$$

The precision of formal languages avoids the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
- ② "*If there is austerity then there is no growth*"
- ③ "*If there is growth then there is no stagnation*"
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
 $a \rightarrow \neg g$
- ② "*If there is austerity then there is no growth*"
- ③ "*If there is growth then there is no stagnation*"
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
 $a \rightarrow \neg g$
- ② "*If there is austerity then there is no growth*"
 $a \rightarrow \neg g$
- ③ "*If there is growth then there is no stagnation*"
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
 $a \rightarrow \neg g$
- ② "*If there is austerity then there is no growth*"
 $a \rightarrow \neg g$
- ③ "*If there is growth then there is no stagnation*"
 $g \rightarrow \neg s$
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"

$$a \rightarrow \neg g$$

- ② "*If there is austerity then there is no growth*"

$$a \rightarrow \neg g$$

- ③ "*If there is growth then there is no stagnation*"

$$g \rightarrow \neg s$$

- ④ "*reckless spending is the opposite of austerity*"

$$r \equiv \neg a$$

Evaluation of a formula as a function

The algorithm for checking whether a formula A is satisfied by an interpretation $\mathcal{I}$ makes it clear the relationship between an interpretation and a formula, and shows that **an interpretation can be extended to a function from the set of formulas to the set $\{\text{True}, \text{False}\}$** :

- $\mathcal{I}(A) = \text{True}$ if $\mathcal{I} \models A$
- $\mathcal{I}(A) = \text{False}$ if $\mathcal{I} \not\models A$

For example, it is easy to see that, if $\mathcal{I}(p) = \text{True}$ and $\mathcal{I}(q) = \text{False}$, then $\mathcal{I}(p \vee q) = \text{True}$. Note that it is common to regard the truth values as numbers, usually with the following convention:

- **True** corresponds to 1
- **False** corresponds to 0

Thus, an interpretation can also be seen as a function from the set of formulas to the set $\{0, 1\}$.

Truth tables

The meaning of the connectives we have considered in propositional logic ($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\equiv$) can be represented as a table, called **truth table**. For example:

Example: truth table for the $\wedge$ binary connective ($n = 2$)

A	B	$A \wedge B$
False	False	False
False	True	False
True	False	False
True	True	True

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

In principle, we could think of other connectives whose semantics is specified through truth tables: if a connective is n -ary (i.e., it applies to n formulas), the truth table has as many rows as the possible vectors of n values, each value being True or False, and $n + 1$ columns, n columns for representing each vector, and one for representing the truth value of the formula if the truth values of the various components are the ones specified by the vector.

Boolean functions

More generally, any **Boolean function** can be specified through a truth table, where a Boolean function is a function of type

$$\{\text{False}, \text{True}\}^n \rightarrow \{\text{False}, \text{True}\}$$

or $\{0, 1\}^n \rightarrow \{0, 1\}$, and the corresponding truth table has again as many rows as the possible vectors of n values, and $n + 1$ columns.

Example of Boolean function f ($n = 3$)

A	B	C	$f(A, B, C)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Boolean functions and connectives

It follows that for any connective, there is a corresponding Boolean function capturing its semantics, and viceversa.

- Boolean functions with **0 argument**:

- ▶ $f_T : \rightarrow 1$ corresponds to the new connective $\top$ (for every $\mathcal{I}$, $\mathcal{I}(\top) = \text{True}$)
- ▶ $f_F : \rightarrow 0$ corresponds to the new connective $\bot$ (for every $\mathcal{I}$, $\mathcal{I}(\bot) = \text{False}$)

- Boolean functions with **1 argument**:

A	f_{id}
0	0
1	1

A	$f_{\neg}$
0	1
1	0

A	$f_{1,1}$
0	1
1	1

A	$f_{0,1}$
0	0
1	0

Note that $f_{\neg}$ corresponds to the connective $\neg$. The other 1-argument functions do not correspond to interesting connectives.

Semantics of unary connectives expressed through Boolean functions

If $\circ$ is a 1-argument connective, and $f_{\circ}$ is the corresponding Boolean function, we have that, for every interpretation $\mathcal{I}$, and every formula A ,

$$\mathcal{I}(\circ A) = f_{\circ}(\mathcal{I}(A)).$$

Boolean functions and connectives

Obviously, for every binary connective, there is a corresponding Boolean function, and viceversa.

- Boolean functions with 2 arguments corresponding to our connectives:

A	B	$f_{\wedge}$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$f_{\vee}$
0	0	0
0	1	1
1	0	1
1	1	1

A	B	$f_{\rightarrow}$
0	0	1
0	1	1
1	0	0
1	1	1

A	B	$f_{\equiv}$
0	0	1
0	1	0
1	0	0
1	1	1

Semantics of binary connectives expressed through Boolean functions

If $\circ$ is a 2-argument connective, and $f_{\circ}$ is the corresponding Boolean function, then for every interpretation $\mathcal{I}$, and every formulas A, B ,

$$\mathcal{I}(A \circ B) = f_{\circ}(\mathcal{I}(A), \mathcal{I}(B)).$$

How many Boolean functions?

How many Boolean functions (or connectives) with n arguments we have?

- $n = 0 \rightarrow 2$

f_T
1

f_F
0

- $n = 1 \rightarrow 4$

A	f_{id}
0	0
1	1

A	$f_{\neg}$
0	1
1	0

A	$f_{1,1}$
0	1
1	1

A	$f_{0,1}$
0	0
1	0

How many Boolean functions?

How many Boolean functions (or connectives) with n arguments we have?

In general, all the truth tables of the various Boolean functions with the same number n of arguments have a number of rows equal to the number of interpretations that we can define on n propositional variables, i.e., 2^n . Two such truth tables differ only in the last columns. Now, since there are 2^{2^n} possible columns of values 0,1 with 2^n rows, it follows that there are 2^{2^n} possible, Boolean functions with n arguments.

- $n = 2 \rightarrow 16$

the number of tables with all possible combinations of $\{0, 1\}$ in 2 columns, i.e., number of tables with $2^2 = 4$ rows, and 2 + 1 columns, i.e., the number of $\{0, 1\}$ vectors of 4 elements, i.e., $2^4 = 16$.

- $n \text{ generic} \rightarrow 2^{2^n}$

the number of tables with all possible combinations of $\{0, 1\}$ in n columns, i.e., number of tables with 2^n rows, and $n + 1$ columns, i.e., the number of $\{0, 1\}$ vectors of 2^n elements, i.e., 2^{2^n} .

Truth tables for formulas

It is immediate to build a **truth table for a formula**: if the formula has n propositional variables, the table has one row for each interpretation of the formula (assigning truth values to all the propositional variables), and $n + 1$ columns, where for a row corresponding to an interpretation $\mathcal{I}$, the last column will contain $\mathcal{I}(A)$.

Example

Truth table for $(F \vee G) \wedge \neg(F \wedge G)$.

F	G	$(F \vee G) \wedge \neg(F \wedge G)$
True	True	False
True	False	True
False	True	True
False	False	False

Note: intuitively, what does the formula in the example represent?

Truth tables for formulas: exercises

Suggestion: to simplify the process of building a truth table for a formula, we can add columns to the table corresponding to the various subformulas

Example: Truth table for $(F \vee G) \wedge \neg(F \wedge G)$

F	G	$F \vee G$	$F \wedge G$	$\neg(F \wedge G)$	$(F \vee G) \wedge \neg(F \wedge G)$
True	True	True	True	False	False
True	False	True	False	True	True
False	True	True	False	True	True
False	False	False	False	True	False

Exercise: build the truth tables for the following propositional formulas:

- $(p \rightarrow p) \rightarrow p$
- $p \rightarrow (p \rightarrow p)$
- $p \vee q \rightarrow p \wedge q$
- $p \vee (q \wedge r) \rightarrow (p \wedge r) \vee q$
- $p \rightarrow (q \rightarrow p)$
- $(p \wedge \neg q) \vee \neg(p \leftrightarrow q)$

From formulas to Boolean functions

It can be shown that there is total correspondence between Boolean functions and formulas. One direction is easy to prove.

Proposition

For every formula A with n propositional variables $p_1, \dots, p_n$, one can define a Boolean function f_A with n arguments such that for every interpretation $\mathcal{I}$, $\mathcal{I}(A) = f_A(\mathcal{I}(p_1), \dots, \mathcal{I}(p_n))$.

The proof of the above proposition is based on the fact that the truth table for a formula directly specifies the corresponding Boolean function.

From Boolean functions to formulas

It can be shown that there is total correspondence between Boolean functions and formulas. The other direction is represented by the following proposition.

Proposition

For every Boolean function f with n arguments, there is a corresponding formula A_f with n propositional variables $p_1, \dots, p_n$, such that for every interpretation $\mathcal{I}$, $\mathcal{I}(A_f) = f(\mathcal{I}(p_1), \dots, \mathcal{I}(p_n))$.

The above proposition is more difficult to prove.

From Boolean functions to formulas: first intuition

Given the truth table of a Boolean function ϕ with n arguments ($n > 0$), we can build the formula with $n + 1$ propositional variables $p_0, p_1, \dots, p_n$ of the form

$$(p_0 \wedge \neg p_0) \vee \alpha_1 \vee \dots \vee \alpha_k$$

with one α_i for each row with value 1 for ϕ , where each α_i has the form

$$\gamma_1 \wedge \dots \wedge \gamma_n$$

where γ_j is p_j if the row has 1 in column j , and is $\neg p_j$ if the row has value 0 in column j .

Note that $(p_0 \wedge \neg p_0)$ takes care of the case where $\phi(p_1, \dots, p_n) = 0$.

Example: formula corresponding to the Boolean function ϕ

p_1	p_2	p_3	$\phi(p_1, p_2, p_3)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0



$$(p_0 \wedge \neg p_0) \vee (\neg p_1 \wedge \neg p_2 \wedge \neg p_3) \vee (\neg p_1 \wedge p_2 \wedge p_3)$$

From Boolean functions to formulas: second intuition

Given the truth table of a Boolean function ψ with n arguments ($n > 0$), we can build the formula with $n + 1$ propositional variables $p_0, p_1, \dots, p_n$ of the form

$$\neg((p_0 \wedge \neg p_0) \vee \beta_1 \vee \dots \vee \beta_k)$$

with one β_i for each row with value 0 for ψ , where each β_i has the form

$$\gamma_1 \wedge \dots \wedge \gamma_n$$

where γ_j is p_j if the row has 1 in column j , and is $\neg p_j$ if the row has value 0 in column j .

Note that $(p_0 \wedge \neg p_0)$ takes care of the case where $\phi(p_1, \dots, p_n) = 1$.

Example: formula corresponding to the Boolean function ψ

p_1	p_2	p_3	$\psi(p_1, p_2, p_3)$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



$$\neg((p_0 \wedge \neg p_0) \vee (\neg p_1 \wedge \neg p_2 \wedge p_3) \vee (p_1 \wedge \neg p_2 \wedge \neg p_3))$$

by De Morgan, equivalent to

$$(p_0 \vee \neg p_0) \wedge (p_1 \vee p_2 \vee \neg p_3) \wedge (\neg p_1 \vee p_2 \vee p_3)$$

Complete sets of connectives

The two intuitions above are at the basis of the proof that every Boolean function can be represented by a formula. Such proof shows that the following proposition is true.

Proposition

Every formula of propositional logic can be written using only the connectives $\neg$, $\vee$ and $\wedge$.

In other words, the set of connectives $\{\neg, \vee, \wedge\}$ is **complete**, because they allow to express any formula that corresponds to a Boolean function.

Note that this implies that the set of our connectives, i.e., $\{\neg, \vee, \wedge, \rightarrow, \equiv\}$ is also complete.

Even more specifically, it can be shown that at least two *normal forms* exist for propositional formulas.

Normal form for formulas

Proposition

Every Boolean function can be expressed by a formula of propositional logic in the so-called *disjunctive normal form*, i.e., as

$$\alpha_1 \vee \cdots \vee \alpha_k$$

where each α_i is a *minterm*, i.e., has the form

$$\gamma_1 \wedge \cdots \wedge \gamma_n$$

where each γ_j is a *literal*, i.e., either a propositional variable or the negation of a propositional variable.

Proposition

Every Boolean function can be expressed by a formula of propositional logic in the so-called *conjunctive normal form*, i.e., as

$$\beta_1 \wedge \cdots \wedge \beta_k$$

where each β_i is a *clause*, i.e., has the form

$$\gamma_1 \vee \cdots \vee \gamma_n$$

where each γ_j is a *literal*.

Are all our connectives necessary?

We leave as an exercise to prove the following propositions, regarding other complete sets of connectives.

Proposition

Every formula of propositional logic can be written using only the connectives $\neg$ and $\vee$.

Proposition

Every formula of propositional logic can be written using only the connectives $\neg$ and $\wedge$.

Also, if we assume we can use also T and F in formulas, the following propositions hold.

Proposition

Every formula of propositional logic can be written using only the connective NAND or only the connective NOR.

Valid, satisfiable, and unsatisfiable formulas

Definition (Model of a formula)

An interpretation $\mathcal{I}$ that satisfies a formula A , i.e., such that $\mathcal{I} \models A$, is called a **model** of A .

Definition

A formula A is

Valid or **tautology** if **for all interpretations** $\mathcal{I}$, $\mathcal{I} \models A$
i.e., every interpretation is a model of A

Satisfiable if **there is an interpretation** $\mathcal{I}$ s.t. $\mathcal{I} \models A$
i.e., A has at least one model

Unsatisfiable if **for no interpretation** $\mathcal{I}$, $\mathcal{I} \models A$
i.e., A has no model

Valid, satisfiable, and unsatisfiable formulas

Proposition

$A \text{ valid} \longrightarrow A \text{ satisfiable} \longleftrightarrow A \text{ not unsatisfiable}$

$A \text{ unsatisfiable} \longleftrightarrow A \text{ not satisfiable} \longrightarrow A \text{ not Valid}$

Proposition

<i>if A is</i>	<i>then $\neg A$ is</i>
<i>Valid</i>	<i>Unsatisfiable</i>
<i>Satisfiable</i>	<i>not Valid</i>
<i>not Valid</i>	<i>Satisfiable</i>
<i>Unsatisfiable</i>	<i>Valid</i>

Checking validity and (un)satisfiability of a formula

Truth table

Checking (un)satisfiability and validity of a formula A can be done by enumerating all the interpretations which are relevant for A , and for each interpretation $\mathcal{I}$ check if $\mathcal{I} \models A$.

Example (using truth table)

A	B	C	$A \rightarrow (B \vee \neg C)$
True	True	True	True
True	True	False	True
True	False	True	False
True	False	False	True
False	True	True	True
False	True	False	True
False	False	True	True
False	False	False	True

Valid, satisfiable, and unsatisfiable formulas

Example

Satisfiable	$A \rightarrow A$	}	Valid
	$A \vee \neg A$		
	$\neg\neg A \equiv A$		
	$\neg(A \wedge \neg A)$		
	$A \wedge B \rightarrow A$		
	$A \rightarrow A \vee B$		
Unsatisfiable	$A \vee B$	}	Non Valid
	$A \rightarrow B$		
	$\neg(A \vee B) \rightarrow C$		
	$A \wedge \neg A$		
	$\neg(A \rightarrow A)$		
	$A \equiv \neg A$		
	$\neg(A \equiv A)$		

Prove that the blue formulas are valid, that the magenta formulas are satisfiable but not valid, and that the red formulas are unsatisfiable.

Valid, satisfiable, and unsatisfiable sets of formulas

Definition (Model of a set of formula)

An interpretation $\mathcal{I}$ that satisfies all the formulas of a set Γ , i.e., such that $\mathcal{I} \models A$ for all formulas $A \in \Gamma$, is called a **model** of Γ (written $\mathcal{I} \models \Gamma$).

Definition

A **set of formulas** Γ is

Valid if for all $\mathcal{I}$, $\mathcal{I} \models \Gamma$, i.e., every interpretation is a model of Γ

Satisfiable if there is an $\mathcal{I}$ s.t. $\mathcal{I} \models \Gamma$, i.e., Γ has at least one model

Unsatisfiable if for no $\mathcal{I}$, $\mathcal{I} \models \Gamma$, i.e., Γ has no model

Proposition

For any **finite set** of formulas Γ , (i.e., $\Gamma = \{A_1, \dots, A_n\}$ for some $n \geq 1$), Γ is valid (resp. satisfiable and unsatisfiable) if and only if $A_1 \wedge \dots \wedge A_n$ is valid (resp. satisfiable and unsatisfiable).

Validity check by truth tables: Example

We use the truth tables method to determine whether $(p \rightarrow q) \vee (p \rightarrow \neg q)$ is valid.

p	q	$p \rightarrow q$	$\neg q$	$p \rightarrow \neg q$	$(p \rightarrow q) \vee (p \rightarrow \neg q)$
True	True	True	False	False	True
True	False	False	True	True	True
False	True	True	False	True	True
False	False	True	True	True	True

The formula is valid since it is satisfied by every interpretation.

Satisfiability check by truth tables: Example

Use the truth tables method to determine whether $(\neg p \vee q) \wedge (q \rightarrow \neg r \wedge \neg p) \wedge (p \vee r)$ (denoted with ϕ) is satisfiable.

p	q	r	$\neg p \vee q$	$\neg r \wedge \neg p$	$q \rightarrow \neg r \wedge \neg p$	$(p \vee r)$	ϕ
True	True	True	True	False	False	True	False
True	True	False	True	False	False	True	False
True	False	True	False	False	True	True	False
True	False	False	False	False	True	True	False
False	True	True	True	False	False	True	False
False	True	False	True	True	True	False	False
False	False	True	True	False	True	True	True
False	False	False	True	True	True	False	False

There exists an interpretation satisfying ϕ , thus ϕ is satisfiable.

Truth tables: Exercises

Use the truth table method to verify whether the following formulas are valid, satisfiable or unsatisfiable:

- $(p \rightarrow q) \wedge \neg q \rightarrow \neg p$
- $(p \rightarrow q) \rightarrow (p \rightarrow \neg q)$
- $(p \vee q \rightarrow r) \vee p \vee q$
- $(p \vee q) \wedge (p \rightarrow r \wedge q) \wedge (q \rightarrow \neg r \wedge p)$
- $(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))$
- $(p \vee q) \wedge (\neg q \wedge \neg p)$
- $(\neg p \rightarrow q) \vee ((p \wedge \neg r) \leftrightarrow q)$
- $(p \rightarrow q) \wedge (p \rightarrow \neg q)$
- $(p \rightarrow (q \vee r)) \vee (r \rightarrow \neg p)$

Logical consequence (or logical implication)

Definition (Logical consequence or logical implication)

A formula A is a **logical consequence** of (or, is **logically implied** by) a set of formulas Γ , denoted by $\Gamma \models A$ if every model of Γ is also a model of A (i.e., for all $\mathcal{I}$, $\mathcal{I} \models \Gamma$ implies $\mathcal{I} \models A$).

We write $\models A$ to mean $\emptyset \models A$, i.e., to mean that A is valid. We write $\Gamma \not\models A$ to mean that A is **not logically implied** by Γ . Thus, $\not\models A$ means A not valid.

Observation

$\Gamma \not\models A$ means that there exists at least one model of Γ where A is false.

If Γ_1 and Γ_2 are sets of formulas, we also define $\Gamma_1 \models \Gamma_2$ simply as $\Gamma_1 \models A$ for every $A \in \Gamma_2$. Thus, $\emptyset \models \Gamma$, $\not\models \Gamma$, and $\Gamma_1 \not\models \Gamma_2$ are defined accordingly.

Overloading of $\models$

Note that the symbol $\models$ is overloaded:

- when $\models$ has an interpretation $\mathcal{I}$ on the left, then it means **satisfaction**, i.e., $\mathcal{I} \models \alpha$ means that $\mathcal{I}$ satisfies α ;
- when $\models$ has a formula Γ on the left, then it means **logical implication**, i.e., $\Gamma \models \alpha$ means that Γ logically implies α .

Logical equivalence

Definition (Logical equivalence)

Two formulas F and G are **logically equivalent** (denoted with $F \models G$) if both $F \models G$ and $G \models F$, i.e., for every interpretation $\mathcal{I}$, $\mathcal{I}(F) = \mathcal{I}(G)$ (in other words, if the set of models of F and G coincide).

Logical equivalence is naturally extended to sets of formulas: $\Gamma_1 \models \Gamma_2$ if both $\Gamma_1 \models \Gamma_2$ and $\Gamma_2 \models \Gamma_1$.

Sometimes, logical equivalence is denoted by the same symbol used for “material equivalence”, i.e., $\equiv$. However, it is important not to confuse the two notions:

- material implication is **in** the language of propositional logic
- logical implication is in the **meta-language**, i.e., it is **outside** the language of propositional logic.

Exercises on logical implication

Check that the following logical implications hold

- $p \models p \vee q$
- $q \vee p \models p \vee q$
- $\{p \vee q, p \rightarrow r, q \rightarrow r\} \models r$
- $\{p \rightarrow q, p\} \models q$

Hint: base your check on the definition of “satisfaction”, i.e., check that every model of the formulas on the left is also a model of the formula on the right.

Solution: checking logical consequence in a direct manner

Verify that $p \models p \vee q$ Suppose that $\mathcal{I} \models p$, then by definition $\mathcal{I} \models p \vee q$.

Verify that $q \vee p \models p \vee q$ Suppose that $\mathcal{I} \models q \vee p$, then $\mathcal{I} \models q$ or $\mathcal{I} \models p$. In both cases we have that $\mathcal{I} \models p \vee q$.

Verify that $\{p \vee q, p \rightarrow r, q \rightarrow r\} \models r$ Suppose that $\mathcal{I} \models p \vee q$ and $\mathcal{I} \models p \rightarrow r$ and $\mathcal{I} \models q \rightarrow r$. Then $\mathcal{I} \models p$ or $\mathcal{I} \models q$. In the first case, since $\mathcal{I} \models p \rightarrow r$, then $\mathcal{I} \models r$. In the second case, since $\mathcal{I} \models q \rightarrow r$, then $\mathcal{I} \models r$.

Verify that $\{p \rightarrow q, p\} \models q$ Suppose that $\mathcal{I} \models p \rightarrow q$ and $\mathcal{I} \models p$. By the semantics of $\rightarrow$, $\mathcal{I} \models p \rightarrow q$ means that $\mathcal{I} \models \neg p \vee q$. Therefore, $\mathcal{I} \models q$.

Logical consequence and equivalence using truth tables

Use the truth tables method to determine whether $\neg p \models p \wedge \neg q \rightarrow p \wedge q$.

p	q	$\neg p$	$p \wedge \neg q$	$p \wedge q$	$p \wedge \neg q \rightarrow p \wedge q$
True	True	False	False	True	True
True	False	False	True	False	False
False	True	True	False	False	True
False	False	True	False	False	True

Use the truth tables method to determine whether $p \rightarrow (q \wedge \neg q) \models \neg p$.

p	q	$q \wedge \neg q$	$p \rightarrow (q \wedge \neg q)$	$\neg p$
True	True	False	False	False
True	False	False	False	False
False	True	False	True	True
False	False	False	True	True

Truth tables: exercises

Use the truth table method to determine whether the following logical consequences and logical equivalences hold:

- $(p \rightarrow q) \models \neg p \rightarrow \neg q$
- $(p \rightarrow q) \wedge \neg q \models \neg p$
- $p \rightarrow q \wedge r \models (p \rightarrow q) \rightarrow r$
- $p \vee (\neg q \wedge r) \models q \vee \neg r \rightarrow p$
- $\neg(p \wedge q) \models \neg p \vee \neg q$
- $(p \vee q) \wedge (\neg p \rightarrow \neg q) \models q$
- $(p \wedge q) \vee r \models (p \rightarrow \neg q) \rightarrow r$
- $(p \vee q) \wedge (\neg p \rightarrow \neg q) \models p$
- $((p \rightarrow q) \rightarrow q) \rightarrow q \models p \rightarrow q$

Properties of propositional logical consequence

Proposition

If Γ and Σ are two sets of propositional formulas, and A and B two formulas, then the following properties hold:

Reflexivity If $A \in \Gamma$, then $\Gamma \models A$

Ex falso sequitur quodlibet If Γ is unsatisfiable, then $\Gamma \models A$ for all A

Monotonicity If $\Gamma \models A$ then $\Gamma \cup \Sigma \models A$

Cut If $\Gamma \models A$ and $\Sigma \cup \{A\} \models B$ then $\Gamma \cup \Sigma \models B$

Compactness If $\Gamma \models A$, then there is a finite subset $\Gamma_0 \subseteq \Gamma$, such that $\Gamma_0 \models A$

Deduction theorem If $\Gamma \cup \{A\} \models B$ then $\Gamma \models A \rightarrow B$

Deduction principle $\Gamma \cup \{A\} \models B$ if and only if $\Gamma \models A \rightarrow B$

Refutation principle $\Gamma \models A$ if and only if $\Gamma \cup \{\neg A\}$ is unsatisfiable

Properties of propositional logical consequence

Reflexivity If $A \in \Gamma$, then $\Gamma \models A$.

PROOF: If $A \in \Gamma$, and $\mathcal{I} \models \Gamma$, then obviously $\mathcal{I} \models A$.

Ex falso sequitur quodlibet If Γ is unsatisfiable, then $\Gamma \models A$ for all A

PROOF: Recall that $\Gamma \not\models A$ means that there is a model of Γ where A is false. But if Γ is unsatisfiable there is no model of Γ , and therefore it cannot be that $\Gamma \not\models A$. Thus we conclude that $\Gamma \models A$.

Monotonicity If $\Gamma \models A$ then $\Gamma \cup \Sigma \models A$

PROOF: Every model of $\Gamma \cup \Sigma$ is clearly a model Γ . Therefore, If $\Gamma \models A$, then every model of $\Gamma \cup \Sigma$ is also a model of A , i.e., $\Gamma \cup \Sigma \models A$.

Cut If $\Gamma \models A$ and $\Sigma \cup \{A\} \models B$ then $\Sigma \cup \Gamma \models B$.

PROOF: For all $\mathcal{I}$, if $\mathcal{I} \models \Sigma \cup \{A\}$, then $\mathcal{I} \models \Sigma$ and $\mathcal{I} \models A$. The hypothesis $\Gamma \models A$ implies that $\mathcal{I} \models A$. Since $\mathcal{I} \models \Sigma$, then $\mathcal{I} \models \Sigma \cup \{A\}$. The hypothesis $\Sigma \cup \{A\} \models B$, implies that $\mathcal{I} \models B$. We can therefore conclude that $\Sigma \cup \Gamma \models B$.

Properties of propositional logical consequence

Compactness If $\Gamma \models A$, then there is a finite subset $\Gamma_0 \subseteq \Gamma$, such that $\Gamma_0 \models A$.

PROOF: Let $\mathcal{P}_A$ be the propositional variables occurring in A . Let $\mathcal{I}_1, \dots, \mathcal{I}_n$ (with $n \leq 2^{|\mathcal{P}_A|}$), be all the interpretations of the language $\mathcal{P}_A$ that do not satisfy A . Since $\Gamma \models A$, then there should be $\mathcal{I}'_1, \dots, \mathcal{I}'_n$ interpretations of the language of Γ , which are extensions of $\mathcal{I}_1, \dots, \mathcal{I}_n$, and such that $\mathcal{I}'_k \not\models \gamma_k$ for some $\gamma_k \in \Gamma$. Let $\Gamma_0 = \{\gamma_1, \dots, \gamma_k\}$. Then $\Gamma_0 \models A$. Indeed if $\mathcal{I} \models \Gamma_0$ then $\mathcal{I}$ is an extension of an interpretation J of $\mathcal{P}_A$ that satisfies A , and therefore $\mathcal{I} \models A$.

Deduction theorem If $\Gamma \cup \{A\} \models B$ then $\Gamma \models A \rightarrow B$

PROOF: Assume $\Gamma \cup \{A\} \models B$. Consider an interpretation $\mathcal{I}$, and suppose that $\mathcal{I} \models \Gamma$. If $\mathcal{I} \not\models A$, then $\mathcal{I} \models A \rightarrow B$. If instead $\mathcal{I} \models A$, then $\mathcal{I} \models \Gamma \cup \{A\}$. But since $\Gamma \cup \{A\} \models B$, we conclude that $\mathcal{I} \models B$. We can therefore conclude that $\Gamma \models A \rightarrow B$.

Properties of propositional logical consequence

Deduction principle $\Gamma \cup \{A\} \models B$ if and only if $\Gamma \models A \rightarrow B$

PROOF: One direction coincides with the deduction theorem. The other direction amount to prove that if $\Gamma \models A \rightarrow B$, then $\Gamma \cup \{A\} \models B$. But $\Gamma \models A \rightarrow B$ means that every model of Γ is a model of $\neg A \vee B$. This means that every model of Γ where A is true must have B true, and thus every model of $\Gamma \cup A$ is a model of B , which means $\Gamma \cup \{A\} \models B$.

Refutation principle $\Gamma \models A$ if and only if $\Gamma \cup \{\neg A\}$ is unsatisfiable

PROOF:

($\implies$) Suppose by contradiction that $\Gamma \cup \{\neg A\}$ is satisfiable. This implies that there is an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \Gamma$ and $\mathcal{I} \models \neg A$, i.e., $\mathcal{I} \not\models A$. This contradicts that fact that for all interpretations that satisfies Γ , they satisfy A

($\impliedby$) Let $\mathcal{I} \models \Gamma$, then by the fact that $\Gamma \cup \{\neg A\}$ is unsatisfiable, we have that $\mathcal{I} \not\models \neg A$, and therefore $\mathcal{I} \models A$. We can conclude that $\Gamma \models A$.

Logic and questions

- When we pose a dichotomic question Q and we implicitly refer to a **specific interpretation** $\mathcal{I}$, the formalization in logic corresponds to checking whether $\mathcal{I} \models Q$. It follows that we are dealing with evaluating a formula (representing a Boolean questions) in an interpretation, and the possible answers are:
 - ▶ **yes**, if $\mathcal{I} \models Q$
 - ▶ **no**, if $\mathcal{I} \not\models Q$

Example: “In March, can you have sun while raining”? **yes**

- When we pose a dichotomic question Q and we refer to a **set of more than one interpretations**, denoted by a set of formulas Γ , the formalization in logic corresponds to checking whether $\Gamma \models Q$. It follows that we are dealing with logical implication, and the possible answers are:
 - ▶ **yes**, if $\Gamma \models Q$
 - ▶ **no**, if $\Gamma \models \neg Q$
 - ▶ **don't know**, if $\Gamma \not\models Q$ and $\Gamma \not\models \neg Q$

Example: “In March or August, can you have sun while raining”? **don't know** (depends on month)

Logical implication and formal reasoning

We discuss two basic forms of reasoning based on semantics, i.e., based on logical implication.

- **Deductive reasoning (deduction) via logical implication**

Check whether a formula A can be **deduced** from a set of formulas Γ , i.e., check whether

$$\Gamma \models A$$

- **Abductive reasoning (abduction) via logical implication**

Check whether a certain formula can be used to deduce another formula A in the context of a background knowledge represented by a set of formulas Γ , i.e., check whether there exists a formula B such that:

- ▶ $\Gamma \cup \{B\}$ is satisfiable, and
- ▶ $\Gamma \cup \{B\} \models A$

In other words, we are checking whether B can be **abduced** from A (or, for explaining A) in the context of Γ .

Logical implication and argumentation

We discuss two basic forms of argumentation and its formalization using logical implication.

- **Support**

To support a sentence S while arguing, one can exhibit a set of sentences (accepted as valid) that logically implies S

Γ supports S if $\Gamma \models S$

Example: “Paolo is adult” is supported by “Paolo has the driver’s license” and “license $\rightarrow$ adult”.

- **Refutation**

To refute a sentence S while arguing, one can exhibit a set of sentences (accepted as valid) that logically implies the negation of S

Γ refutes S if $\Gamma \models \neg S$

Example: “Paolo is adult” is refuted by “Paolo has the driver’s license”, “license $\rightarrow$ adult”, “adult $\rightarrow \neg$ underage”.